2. posAtSource είναι η θέση στον πίνακα source από την οποία επιθυμούμε να αρχίσει η αντιγραφή.
3. destination είναι ο πίνακας-αντίγραφο που θέλουμε να ενημερώσουμε.
4. posAtDest είναι η θέση στον πίνακα destination από όπου επιθυμούμε να αρχίσει η επικόλληση των στοιχείων του source.
5. len είναι το πλήθος των στοιχείων που επιθυμούμε να αντιγράψουμε.

Ο τύπος της πρώτης και της τρίτης παραμέτρου είναι Object. Ο τύπος αυτός παρουσιάζεται εκτεταμένα στην ενότητα 13.1.4. Για την ώρα, αρκεί να γνωρίζουμε ότι κατά την κλήση της arraycopy, μπορούμε να περάσουμε πίνακες στη θέση των παραμέτρων τύπου Object.

```
1   public static void arrayCopyDemo() { //Κώδικας 6.7
2       char[] copyFrom = {'d', 'e', 'c', 'a', 'f', 'f', 'e', 'i', 'n', 'a',
't', 'e', 'd'};
3       char[] copyTo = new char[7];
4        System.arraycopy(copyFrom, 2, copyTo, 0, copyTo.length);
5        System.out.println(new String(copyTo));
    }
```

***Κώδικας 6.7*** *Παράδειγμα χρήσης της arraycopy*

Στη γραμμή 2 του κώδικα 6.7 ορίζουμε τον πίνακα χαρακτήρων copyFrom. Όπως φανερώνει το όνομά του είναι ο πίνακας που θα αντιγραφεί και πιο συγκεκριμένα μέρος του οποίου θα αντιγράψουμε στον πίνακα copyTo που δηλώνεται και δημιουργείται στη γραμμή 3. Προσέξτε πως το μήκος του copyFrom είναι 13, ενώ του copyTo είναι 7. Η επιλογή αυτή οφείλεται στην πρόθεσή μας να αντιγράψουμε μόνο ένα τμήμα του copyFrom. Αν θέλουμε να αντιγράψουμε ολόκληρο τον πίνακα, τότε ο πίνακας προορισμού πρέπει να δημιουργηθεί σύμφωνα με το μήκος του πίνακα πηγής. Στη γραμμή 4 καλούμε την arraycopy για να πραγματοποιήσουμε την αντιγραφή. Στην κλήση αυτή περνάμε ως πρώτο όρισμα τον copyFrom και ως τρίτο τον copyTo. Το δεύτερο όρισμα σηματοδοτεί τη θέση στον copyFrom στην οποία αρχίζουν τα δεδομένα που θέλουμε να αντιγράψουμε. Εφόσον έχουμε περάσει την τιμή 2, τα δεδομένα θα αρχίσουν να αντιγράφονται από τη δεύτερη θέση του copyFrom, δηλαδή από τον χαρακτήρα ‘c’. Το τελευταίο όρισμα έχει ως αποτέλεσμα το πλήθος των χαρακτήρων που θα αντιγραφεί να είναι 7. Επομένως, στον copyTo θα αντιγραφούν οι χαρακτήρες 'c', 'a', 'f', 'f', 'e', 'i', 'n'. Τέλος, στη γραμμή 5 χρησιμοποιούμε μια βολική μέθοδο για να εκτυπώσουμε τον πίνακα χαρακτήρων copyTo. Αντί να εκτυπώσουμε έναν-έναν τους χαρακτήρες του, προφανώς με έναν κατάλληλο βρόχο, δημιουργούμε ένα String με όρισμα τον copyTo. Όπως ήδη γνωρίζουμε, η print γνωρίζει πώς ακριβώς να εμφανίσει δεδομένα τύπου String.

Σημαντικό είναι να προσέξουμε πως κατά τη χρήση της arraycopy πρέπει να δημιουργήσουμε έναν πίνακα με επαρκές μέγεθος για τα δεδομένα που θα αντιγραφούν. Στην περίπτωση που στη θέση του destination περάσουμε πίνακα μικρότερου μήκους από τα δεδομένα που θα αντιγραφούν θα προκληθεί σφάλμα χρόνου εκτέλεσης.

Ας δούμε τώρα την copyOfRange. Η διεπαφή (interface) της συνάρτησης για πίνακες ακέραιων έχει ως εξής:

```
public static int[] copyOfRange(int[] original, int from,
int to)
```

Το όρισμα int[] σημαίνει πως στη θέση εκείνη μπορούμε να περάσουμε οποιονδήποτε μονοδιάστατο πίνακα ακέραιων. Επίσης, το int[] πριν το όνομα της συνάρτησης σημαίνει πως η συνάρτηση θα επιστρέψει στο περιβάλλον κλήσης έναν μονοδιάστατο πίνακα ακέραιων.

Η συνάρτηση είναι υπερφορτωμένη (overloaded) για όλους τους θεμελιώδεις τύπους. Αυτό σημαίνει πως στη θέση του πίνακα ακέραιων μπορούμε να έχουμε πίνακα οποιουδήποτε θεμελιώδη τύπου. Περισσότερες λεπτομέρειες για την υπερφόρτωση συναρτήσεων στην ενότητα 7.2.

Επίσης, υποστηρίζεται μια γενίκευση (generic) της συνάρτησης η οποία μας επιτρέπει να αντιγράφουμε και πίνακες μη θεμελιωδών τύπων. Περισσότερα για τις γενικεύσεις στην ενότητα 17.

Συνοψίζοντας, σημειώνουμε πως στην copyOfRange μπορούμε να περνάμε πίνακες οποιουδήποτε τύπου. Πάντα θα μας επιστρέψει έναν πίνακα ίδιου τύπου με αυτόν που δώσαμε ως όρισμα κατά την κλήση της.

Τα ορίσματα from και to καθορίζουν από ποιο στοιχείο θα αρχίσει η αντιγραφή και σε ποιο στοιχείο θα καταλήξει. Ωστόσο, να δοθεί προσοχή στο γεγονός ότι το τελευταίο στοιχείο που θα αντιγραφεί είναι το στοιχείο στη θέση to-1 και όχι το στοιχείο στη θέση to.

```
    public static void copyOfRangeDemo() { //Κώδικας 6.8
        char[] copyFrom = {'d', 'e', 'c', 'a', 'f', 'f', 'e', 'i', 'n', 'a',
't', 'e', 'd'};
        char[] copyTo = Arrays.copyOfRange(copyFrom, 2, 9);
        System.out.println(new String(copyTo));
    }
```

***Κώδικας 6.8*** *Παράδειγμα χρήσης της copyOfRange*

Ο κώδικας 6.8 παρουσιάζει ένα παράδειγμα χρήσης της copyOfRange. Να υπογραμμίσουμε πως για να χρησιμοποιηθεί πρέπει να γίνει εισαγωγή της κλάσης Java.util.Arrays. Αυτό επιτυγχάνεται με την πρόταση import Java.util.Arrays; αμέσως μετά τη δήλωση του πακέτου. Περισσότερα για τη διαδικασία import στην ενότητα 7.8.

Αρχικά ορίζουμε τον πίνακα χαρακτήρων copyFrom. Στη συνέχεια καλούμε την copyOfRange με ορίσματα τον copyFrom, το 2 και το 9. Η copyOfRange θα δημιουργήσει και θα επιστρέψει έναν πίνακα χαρακτήρων με δεδομένα από τη θέση 2 του copyFrom, δηλαδή από τον χαρακτήρα 'c' έως και τη θέση 8, δηλαδή τον χαρακτήρα 'n'. Επομένως, η copyOfRangeDemo θα εμφανίσει τη σειρά "caffein".

Προσέξτε ότι κατά τη χρήση της copyOfRange δεν χρειάστηκε να δημιουργήσουμε πίνακα στον οποίο θα αντιγράφονταν τα δεδομένα. Τον πίνακα αυτόν δημιούργησε η copyOfRange και τον επέστρεψε στο πρόγραμμά μας. Επομένως, κατά τη χρήση της copyOfRange δεν κινδυνεύουμε να κάνουμε λάθος με το μέγεθος του πίνακα-αντιγράφου όπως στην arraycopy.

## 6.2 Η κλάση Arrays

Εκτός από την copyOfRange, η κλάση Arrays περιλαμβάνει μια σειρά από στατικές συναρτήσεις που υποστηρίζουν ποικιλία επεξεργασιών των πινάκων. Παρόμοια με την copyOfRange, οι συναρτήσεις που συζητάμε εδώ είναι οργανωμένες κατά τέτοιο τρόπο ώστε στις περισσότερες περιπτώσεις να επιτρέπουν την επεξεργασία πίνακα οποιουδήποτε τύπου. Σε αυτήν την ενότητα θα παρουσιάσουμε μέρος των συναρτήσεων της Arrays. Η διεπαφή των συναρτήσεων παρουσιάζεται για πίνακες ακέραιων αλλά αντίστοιχα ισχύουν και για τους υπόλοιπους τύπους. Όπου υπάρχουν διαφορές επισημαίνονται σαφώς.

### 6.2.1 Ταξινόμηση

Όλοι οι θεμελιώδεις τύποι πλην του τύπου boolean έχουν στην Java μια φυσική σειρά (natural order), δηλαδή για 2 τιμές ενός τύπου είναι προκαθορισμένο ποια είναι μεγαλύτερη και ποια μικρότερη. Οι τιμές boolean θεωρείται πως δεν διαθέτουν φυσική σειρά, δηλαδή δεν θεωρείται εξ ορισμού πως το false είναι μικρότερο από το true ή το αντίθετο.

Η Java υποστηρίζει ταξινόμηση πινάκων όλων των θεμελιωδών, πλην του boolean, τύπων καθώς και των μη θεμελιωδών τύπων. Ωστόσο, για πολλούς μη θεμελιώδεις τύπους δεν υπάρχει φυσική σειρά. Αντίθετα, η σειρά των μη θεμελιωδών τύπων πρέπει να καθοριστεί με ευθύνη του προγραμματιστή. Το θέμα αυτό θα συζητήσουμε στην ενότητα 14.1.2. Η συζήτηση εδώ θα περιοριστεί στην ταξινόμηση πινάκων θεμελιωδών τύπων. Η διεπαφή της συνάρτησης ταξινόμησης για πίνακες ακέραιων είναι:

```
public static void sort(int[] a)
```

Επομένως, αν επιθυμούμε να ταξινομήσουμε έναν πίνακα ακέραιων t, αρκεί η κλήση:

```
Arrays.sort(t);
```

Ίδια είναι η κλήση και για οποιονδήποτε άλλο θεμελιώδη τύπο, πλην του τύπου boolean.

### 6.2.2 Δυαδική αναζήτηση (Binary Search)

Κατά την αναζήτηση, ελέγχουμε έναν πίνακα για να διαπιστώσουμε αν περιλαμβάνει ή όχι μια τιμή. Αν η τιμή βρεθεί, επιστρέφεται η θέση της στον πίνακα. Αν η τιμή δεν βρεθεί, επιστρέφεται -1. Ειδικότερα, η δυαδική αναζήτηση εφαρμόζεται μόνο σε ταξινομημένους πίνακες. Η εφαρμογή της σε μη ταξινομημένους έχει απροσδιόριστα αποτελέσματα.
Η διεπαφή της συνάρτησης δυαδικής αναζήτησης για πίνακες ακέραιων έχει ως εξής:

```
public static int binarySearch(int[] a, int key)
```

Η binarySearch λαμβάνει ως παραμέτρους τον πίνακα στον οποίο επιθυμούμε να πραγματοποιήσουμε αναζήτηση και την τιμή που αναζητούμε. Επιστρέφει τη θέση στον πίνακα του στοιχείου που αναζητούμε ή -1 σε περίπτωση που το στοιχείο δεν υπάρχει στον πίνακα. Επομένως, αν έχουμε έναν μονοδιάστατο ταξινομημένο πίνακα ακέραιων, έστω t, και ψάχνουμε για το στοιχείο schElement, η κλήση θα πρέπει να είναι του τύπου:

```
int idx=Arrays.binarySearch(t, schElement);
```

### 6.2.3 Έλεγχος Ισότητας

Δύο πίνακες θεωρούνται ίσοι μεταξύ τους αν περιέχουν τα ίδια στοιχεία και με την ίδια σειρά.
Η διεπαφή της συνάρτησης ισότητας για πίνακες ακέραιων έχει ως εξής:

```
public static boolean equals(int[] a, int[] a2)
```

Επομένως, αν έχουμε τους πίνακες t1 και t2 και θέλουμε να τους συγκρίνουμε για ισότητα αρκεί η κλήση:

```
boolean tEquals=Arrays.equals(t1, t2);
```

### 6.2.4 Αρχικοποίηση

Η αρχικοποίηση πινάκων με την ίδια τιμή για κάθε στοιχείο του πίνακα γίνεται με τις συναρτήσεις fill. Η διεπαφή της fill για πίνακες ακέραιων έχει ως εξής:

```
public static void fill(int[] a, int val)
```

Επομένως, για να αρχικοποιήσουμε όλα τα στοιχεία ενός πίνακα ακέραιων, t, στην τιμή 1, αρκεί η κλήση

```
Arrays.fill(t,1);
```

### 6.2.5 Αναπαράσταση πίνακα ως String

Αν χρειαζόμαστε αναπαράσταση ενός πίνακα ως αλφαριθμητική σειρά μπορούμε να χρησιμοποιήσουμε την ομάδα συναρτήσεων toString. Η διεπαφή της συνάρτησης για πίνακες ακέραιων έχει ως εξής:

```
public static String toString(int[] a)
```

Για παράδειγμα, αν θέλουμε να τυπώσουμε τον πίνακα ακέραιων t, αρκεί ο ακόλουθος κώδικας:

```
System.out.println(Arrays.toString(t));
```

## 6.3 Πολυδιάστατοι πίνακες

Όπως αναφέραμε στην αρχή αυτού του κεφαλαίου, η Java υποστηρίζει και πολυδιάστατους πίνακες. Θα παρουσιάσουμε εδώ κυρίως τους δισδιάστατους πίνακες ενώ θα δώσουμε και παραδείγματα χρήσης πινάκων με περισσότερες διαστάσεις. Πάντως, ό,τι ισχύει για τους δισδιάστατους πίνακες, ισχύει αναλογικά και για πίνακες περισσότερων διαστάσεων.
Ένας πίνακας δύο διαστάσεων ενός τύπου στην Java είναι ένας πίνακας κάθε στοιχείο του οποίου είναι ένας μονοδιάστατος πίνακας του ίδιου τύπου.

```
static void twoDimArray() { //Κώδικας 6.9
    int[][] store = {
        {1, 2, 3, 4},
        {5, 6, 7},
        {8, 9, 10, 11, 12}
    };
    System.out.println(Arrays.toString(store[0]));
    System.out.println(Arrays.toString(store[1]));
    System.out.println(Arrays.toString(store[2]));
}
```

***Κώδικας 6.9*** *Παράδειγμα δήλωσης και αρχικοποίησης πίνακα δύο διαστάσεων με αρχικοποιητή*

Στο παράδειγμα του κώδικα 6.9 δηλώνουμε και αρχικοποιούμε τον δισδιάστατο πίνακα ακέραιων store. Ο πίνακας έχει δηλωθεί ως δισδιάστατος, καθώς τη δήλωση του τύπου int ακολουθούν δύο σύνολα από τετραγωνικές παρενθέσεις. Αντίστοιχα, μπορούμε να δηλώσουμε τρισδιάστατους πίνακες χρησιμοποιώντας τρία σύνολα τετραγωνικών παρενθέσεων, τετραδιάστατους με τέσσερα σύνολα, κοκ, χωρίς άνω όριο διαστάσεων. Στο συγκεκριμένο παράδειγμα, ο πίνακας store αρχικοποιήθηκε με τη χρήση αρχικοποιητή.
Τα στοιχεία του store είναι τρεις μονοδιάστατοι πίνακες ακέραιων. Επομένως, το μήκος του store είναι τρία, store.length==3. Τα δε στοιχεία του είναι τα εξής: το store[0] είναι ο πίνακας {1, 2, 3, 4}, το store[1] είναι ο πίνακας {5, 6, 7} και το store[2] είναι ο πίνακας {8, 9, 10, 11, 12}. Προσέξτε πως κάθε ένας από τους πίνακες-στοιχεία του store έχει διαφορετικό μέγεθος.

```
static void createTwoDimWithNew() { //Κώδικας 6.10
    int[][] store = new int[3][];
    store[0] = new int[4];
    store[1] = new int[3];
    store[2] = new int[5];
    int data = 0;
    for (int i = 0; i < store.length; i++) {
        for (int j = 0; j < store[i].length; j++) {
            store[i][j] = 1 + data++;
        }
    }
    System.out.println(Arrays.toString(store[0]));
    System.out.println(Arrays.toString(store[1]));
    System.out.println(Arrays.toString(store[2]));
}
```

***Κώδικας 6.10*** *Παράδειγμα δήλωσης και αρχικοποίησης πίνακα δύο διαστάσεων με χρήση της new*

Στον κώδικα 6.10 δημιουργούμε έναν δισδιάστατο πίνακα με τη βοήθεια της new. Στην περίπτωση αυτή, η μία διάσταση, αυτή που καθορίζει τα στοιχεία του store, υποχρεωτικά δηλώνεται μέσω μιας σταθεράς. Η δεύτερη μπορεί να δηλωθεί ή να μην δηλωθεί. Αν δηλωθεί όμως, τότε υποχρεωτικά όλα τα στοιχεία-πίνακες του store θα έχουν τη διάσταση που δηλώθηκε κατά την εκτέλεση της new. Στο παράδειγμά μας, δεν έχουμε δηλώσει τη δεύτερη διάσταση. Έτσι μπορούμε να δημιουργήσουμε στοιχεία-πίνακες του store ποικίλων μεγεθών. Στην περίπτωση αυτή, τα στοιχεία του store πρέπει να δημιουργηθούν ένα προς ένα με χρήση της new. Για τα στοιχεία αυτά δεν είναι δυνατή η χρήση αρχικοποιητή, καθώς ο αρχικοποιητής είναι αποδεκτός μόνο στη γραμμή δήλωσης ενός πίνακα.

Στη συνέχεια, στον κώδικα 6.10, προσπελαύνουμε ένα προς ένα τα τελικά στοιχεία του store με μια εμφωλευμένη for προκειμένου να τους καταχωρήσουμε τιμές. Προσέξτε πως η εσωτερική for πηγαίνει μέχρι store[i].length. Συχνά παρατηρείται σε κώδικες αρχαρίων να χρησιμοποιείται το μήκος του δισδιάστατου πίνακα ως άνω όριο της εσωτερικής for. Αυτό είναι προφανώς λανθασμένη πρακτική, καθώς είναι δυνατό ο δισδιάστατος πίνακας να έχει διαφορετικό μήκος από τα στοιχεία-πίνακες που περιέχει.

Αν όμως όλα τα στοιχεία-πίνακες ενός δισδιάστατου πίνακα έχουν την ίδια διάσταση, τότε αν την δηλώσουμε, τα στοιχεία-πίνακες δημιουργούνται αυτόματα χωρίς τη χρήση της new για καθένα από αυτά.

```
static void createArrayWithCommonLength() { //Κώδικας 6.11
    int[][] s = new int[3][3];
    int data = 0;
    for (int i = 0; i < s.length; i++) {
        for (int j = 0; j < s[i].length; j++) {
            s[i][j] = 1 + data++;
        }
    }
    System.out.println(Arrays.toString(s[0]));
    System.out.println(Arrays.toString(s[1]));
    System.out.println(Arrays.toString(s[2]));
}
```

***Κώδικας 6.11*** *Δημιουργία δισδιάστατου πίνακα με σταθερό μήκος στοιχείων*

Στον κώδικα 6.11 τα στοιχεία του δισδιάστατου πίνακα s έχουν όλα το ίδιο μήκος. Σε αυτήν την περίπτωση κατά τη δημιουργία του s δηλώνουμε και τη δεύτερη διάσταση, οπότε ο s και τα στοιχεία του δημιουργούνται χωρίς να απαιτείται η αρχικοποιητή χρήση της new για κάθε στοιχείο ξεχωριστά.

Θα θεωρήσει κανείς πως σε αυτήν την περίπτωση δεν είναι αναγκαίο η εσωτερική for να αναφέρεται στο μήκος του τρέχοντος στοιχείου. Μια και το μήκος κάθε στοιχείου είναι ίδιο με το μήκος του s μπορεί να χρησιμοποιηθεί το μήκος του s και στην εσωτερική for. Αυτή η προσέγγιση είναι λανθασμένη. Η κατάσταση του πίνακα ανεξάρτητα από το πώς δημιουργήθηκε δεν είναι σταθερή. Είναι πιθανό να έχουν μεσολαβήσει μεταβολές από τη δημιουργία του μέχρι την επεξεργασία του. Για παράδειγμα, μετά τη δημιουργία του s είναι δυνατό να τρέξει κώδικας της μορφής s[1]=new int[7];. Ο κώδικας αυτός μεταβάλλει τον πίνακα στη θέση 1 του s. Για αυτόν τον λόγο, σε όλες τις περιπτώσεις, θα πρέπει να θυμόμαστε πως το μήκος ενός πίνακα, αξιόπιστα, το γνωρίζει μόνο ο ίδιος ο πίνακας.

Σε ποιες περιπτώσεις μπορούμε να αξιοποιήσουμε έναν πίνακα τριών διαστάσεων; Εξαρτάται από τα δεδομένα του προβλήματος που έχουμε να κωδικοποιήσουμε. Έστω, ότι έχουμε δεδομένα σχετικά με τις ώρες απασχόλησης ενός εργαζόμενου. Ας υποθέσουμε πως τα δεδομένα μας είναι δομημένα σε τέσσερις χρονικές περιόδους κάθε μία από τις οποίες αποτελείται από τρεις μήνες, κάθε ένας από τους οποίους περιλαμβάνει είκοσι δύο εργάσιμες ημέρες. Για κάθε μία από αυτές τις ημέρες, ο εργαζόμενος έχει εργαστεί έναν ορισμένο αριθμό ωρών. Τέτοιου τύπου δεδομένα είναι πολύ βολικό να φορτωθούν σε έναν τρισδιάστατο πίνακα. Για παράδειγμα, έστω ο πίνακας

```
int[][][] timesheet=new int[4][3][22];
```

Η πρώτη διάσταση αφορά τις περιόδους, η δεύτερη τον μήνα μιας περιόδου και η τρίτη, τις εργάσιμες ημέρες ενός μήνα. Επομένως, αν θέλουμε να προσπελάσουμε πόσες ώρες εργάστηκε ο εργαζόμενος την έβδομη ημέρα του δεύτερου μήνα της πρώτης περιόδου αρκεί να προσπελάσουμε το στοιχείο timesheet[0][1][6].

## 6.4 Βασικές επεξεργασίες

Συχνά απαιτείται να κάνουμε κάποιες βασικές εργασίες με πίνακες όπως να υπολογίσουμε το άθροισμα των στοιχείων ενός πίνακα αριθμών, να εντοπίσουμε το μεγαλύτερο ή το μικρότερο στοιχείο ενός πίνακα, να πραγματοποιήσουμε σειριακή αναζήτηση ψάχνοντας για μια συγκεκριμένη τιμή σε έναν πίνακα, να ανακατέψουμε τα στοιχεία ενός πίνακα, να γεμίσουμε έναν πίνακα με τυχαίες τιμές. Σε αυτήν την ενότητα, θα δούμε τους κώδικες που μπορούν να αξιοποιηθούν για την υλοποίηση των αναφερόμενων επεξεργασιών.

### 6.4.1 Σειριακή αναζήτηση

Η σειριακή αναζήτηση σε αντίθεση με τη δυαδική μπορεί να εφαρμοστεί σε αταξινόμητους πίνακες. Όπως φανερώνει το όνομά της, ελέγχει ένα προς ένα τα στοιχεία του πίνακα συγκρίνοντάς τα με το στοιχείο που ψάχνουμε.
Ο κώδικας της σειριακής αναζήτησης είναι σχετικά απλός:

```
    public static void sequentialSearch() { //Κώδικας 6.12
        final int[] store = {4, 7, 9, 13};
        int schElement = 7;
        int idx = -1;
        for (int i = 0; i < store.length; i++) {
            if (store[i] == schElement) {
                idx = i;
                break;
            }
        }
        if (idx >= 0) {
            System.out.println("Ο ακέραιος " + schElement + " βρέθηκε στην
θέση " + idx);
        } else {
            System.out.println("Ο ακέραιος " + schElement + " δεν βρέθηκε");
        }
    }
```

***Κώδικας 6.12*** *Σειριακή αναζήτηση σε μονοδιάστατο πίνακα ακέραιων*

Στον κώδικα 6.12 ορίζουμε καταρχάς τον πίνακα ακέραιων store και το στοιχείο για το οποίο θα ψάξουμε, το schElement. Επίσης, ορίζουμε μια ακέραιη μεταβλητή idx που αρχικοποιούμε στο -1. Αν κατά την αναζήτηση στον πίνακα βρεθεί το schElement, τότε στην idx θα εκχωρήσουμε τη θέση του στον πίνακα, οπότε η idx θα αποκτήσει τιμή μεγαλύτερη ή ίση με το 0. Έτσι θα γνωρίζουμε πως το στοιχείο βρέθηκε αλλά και πού ακριβώς είναι τοποθετημένο. Αν με το τέλος της αναζήτησης το στοιχείο δεν έχει βρεθεί, η idx θα παραμείνει μικρότερη από το μηδέν σηματοδοτώντας την αποτυχία εύρεσης του στοιχείου. Όπως έχουμε ήδη δει, η σύμβαση σύμφωνα με την οποία επιστρέφεται -1 για ένα στοιχείο που δεν βρέθηκε σε ένα πίνακα χρησιμοποιείται και από τη συνάρτηση binarySearch της κλάσης Arrays που παρουσιάστηκε στην 6.2.2.

Αφού λοιπόν ορίσουμε την idx μπαίνουμε σε έναν βρόχο for με τη βοήθεια του οποίου ελέγχουμε ένα προς ένα τα στοιχεία του store συγκρίνοντάς τα με το schElement. Αν για κάποιο από αυτά επιβεβαιωθεί ισότητα, τότε το schElement βρέθηκε, οπότε ενημερώνεται η idx και διακόπτεται η επαναληπτική διαδικασία.

Σε μια τέτοια συνάρτηση θα πρέπει να προσέξουμε ιδιαίτερα τον τύπο του πίνακα. Το πρότυπο του κώδικα 6.12 είναι κατάλληλο για πίνακες θεμελιωδών τύπων, όχι όμως και για πίνακες μη θεμελιωδών τύπων όπως ο τύπος String. Στην περίπτωση αυτή ο τελεστής ισότητας == θα πρέπει να αντικατασταθεί με τη συνάρτηση equals του τύπου του πίνακα όπως δείχνει ο κώδικας 6.13:

```
    public static void sequentialSearchReference() { //Κώδικας 6.13
        final String[] store = {"John", "Mary", "Jim", "Kelly"};
        String schElement = new String("Jim");
        int idx = -1;
        for (int i = 0; i < store.length; i++) {
            if (store[i].equals(schElement)) {
                idx = i;
                break;
            }
        }
        if (idx >= 0) {
            System.out.println("Το όνομα " + schElement + " βρέθηκε στην θέση
" + idx);
        } else {
            System.out.println("Το όνομα " + schElement + " δεν βρέθηκε");
        }
    }
```

***Κώδικας 6.13*** *Σειριακή αναζήτηση σε πίνακα αλφαριθμητικών σειρών*

## 6.4.2 Μέγιστο ή ελάχιστο στοιχείο

Ένα άλλο πρόβλημα είναι η αναζήτηση του μέγιστου στοιχείου ενός πίνακα. Πρόκειται για απλή επεξεργασία. Παραθέτουμε τον κώδικα υλοποίησης και στη συνέχεια τον σχολιάζουμε:

```
    public static void max() { //Κώδικας 6.14
        int[] t = {2, 6, 1, 9, 15};
        int max = t[0];
        int idx = 0;
        for (int i = 1; i < t.length; i++) {
            if (t[i] > max) {
                max = t[i];
                idx = i;
            }
        }
        System.out.println("Το μεγαλύτερο στοιχείο είναι το " + max + " και
βρίσκεται στην θέση " + idx);
```

***Κώδικας 6.14*** *Εύρεση του μεγαλύτερου στοιχείου μονοδιάστατου πίνακα*

Στον κώδικα 6.14 καταρχάς ορίζουμε τον πίνακα t. Στη συνέχεια, δύο βοηθητικές μεταβλητές, την max και την idx. Η max αρχικοποιείται στην τιμή του στοιχείου στη θέση μηδέν του πίνακα. Επίσης, η idx αρχικοποιείται ώστε να δείχνει στη θέση μηδέν. Ακολούθως, προσπελαύνουμε τα στοιχεία του πίνακα ένα προς ένα και όποτε βρίσκουμε τιμή μεγαλύτερη από την τρέχουσα τιμή του max ενημερώνουμε την max και την idx. Στο τέλος της διαδικασίας, η max έχει τη μεγαλύτερη τιμή του πίνακα και η idx τη θέση της.

Προσέξτε πως ο βρόχος for ξεκινάει από 1 και όχι από 0. Πράγματι, στο συγκεκριμένο πρόβλημα δεν υπάρχει λόγος να ξεκινήσει η σύγκριση από το 0. Αντίθετα, αν ξεκινούσε από το 0, κατά το πρώτο επαναληπτικό βήμα θα συγκρινόταν η max με το στοιχείο t[0]. Η max όμως σε αυτό το βήμα έχει τιμή ίση με το t[0], οπότε είναι περιττή η σύγκριση.

Στην περίπτωση που ο πίνακας έχει δύο στοιχεία ίσα μεταξύ τους και μεγαλύτερα από τα υπόλοιπα στοιχεία του πίνακα, ο κώδικας 6.14 εντοπίζει το στοιχείο στη μικρότερη θέση. Αν επιθυμούμε να βρούμε το στοιχείο στη μεγαλύτερη θέση αρκεί να τροποποιήσουμε τη συνθήκη της if σε t[i]>=max.

Αντίστοιχα, η εύρεση του μικρότερου στοιχείου, βρίσκεται αν αντικαταστήσουμε τη συνθήκη ελέγχου της if με την έκφραση t[i]<min ή t[i]<=min.

### 6.4.3 Άθροισμα

Ένα άλλο τυπικό πρόβλημα είναι ο υπολογισμός του αθροίσματος των στοιχείων ενός πίνακα. Πρόκειται επίσης για μια σειριακή προσπέλαση των στοιχείων του πίνακα και προσαύξηση ενός αθροιστή κατά την τιμή κάθε στοιχείου. Ο κώδικας 6.15 παρουσιάζει τη λύση.

```
public static void sum() { //Κώδικας 6.15
    int[] t = {2, 6, 1, 9, 15};
    int sum = 0;
    for (int i = 0; i < t.length; i++) {
        sum += t[i];
    }
    System.out.println("Το άθροισμα είναι " + sum);
}
```

***Κώδικας 6.15*** *Άθροισμα στοιχείων μονοδιάστατου πίνακα*

### 6.4.4 Ανακατανομή

Σε πολλές εφαρμογές απαιτείται τυχαία ανακατανομή των στοιχείων ενός πίνακα. Για κάθε στοιχείο του πίνακα, υπολογίζουμε μια τυχαία θέση μέσα στα όρια του πίνακα και αντιμεταθέτουμε το στοιχείο στην τρέχουσα θέση με το στοιχείο στην τυχαία θέση.

```
public static void shuffle() { //Κώδικας 6.16
    int[] t = {2, 6, 1, 9, 15};
    Arrays.sort(t);
    System.out.println(Arrays.toString(t));
    Random r = new Random();

    for (int i = 0; i < t.length; i++) {
        int rIdx = r.nextInt(t.length);
        int tmp = t[i];
        t[i] = t[rIdx];
        t[rIdx] = tmp;
    }
    System.out.println(Arrays.toString(t));
}
```

***Κώδικας 6.16*** *Ανακατανομή στοιχείων πίνακα*

Στον κώδικα 6.16 δημιουργούμε το πίνακα t, τον ταξινομούμε και τον τυπώνουμε έτσι ώστε να είναι εμφανής η διαφορά από τον ανακατεμένο πίνακα. Στη συνέχεια υλοποιείται ο αλγόριθμος ανακατανομής που περιγράφηκε αμέσως προηγουμένως.

### 6.4.5 Αρχικοποίηση με τυχαίες τιμές

Πολλές φορές χρειαζόμαστε να γεμίσουμε έναν πίνακα με τυχαίες τιμές είτε για λόγους ελέγχου είτε επειδή η εφαρμογή μας το απαιτεί. Στον κώδικα 6.17 ο πίνακας γεμίζει με τυχαίες τιμές από το 0 έως 99.

```
public static void fillRandom() { //Κώδικας 6.17
    int[] t = new int[20];
```